Automatic scoring of the Rey-Osterrieth Complex Figure Test using topological features

Kirti Jha

M.Sc. Data Science, Universität Erlangen-Nürnberg (FAU), Erlangen
kirti.jha@fau.de

1 Introduction

In today's world, there are varied kinds of diseases and disorders that we come across in our day-to-day lives. One such category of disorders is neuropsychological. These disorders refer to a set of mental illnesses related to brain functioning and behavior. They include conditions such as dementia, Alzheimer's disease, Parkinson's disease, Huntington's disease, attention-deficit hyperactivity disorder (ADHD), schizophrenia, and bipolar disorder. These conditions can cause significant impairment in thinking, planning, and memory, as well as emotional, social, and physical distress [1].

Neuropsychological disorders are often complex and difficult to diagnose and treat, so early detection and intervention are essential for maintaining good mental health. Treatment may involve psychotherapy, medications, or a combination of the two. Early recognition and management of neuropsychological disorders can help reduce the risks of further deterioration and improve the quality of life for those affected [1].

The Rey-Osterrieth Complex Figure (ROCF) test is a popular neuropsychological evaluation tool. The Swiss psychologist André Rey developed the Rey-Osterrieth complex figure exam (ROCFT) in 1941, and Paul-Alexandre Osterrieth standardized it in 1944. To pass the exam, participants must replicate a complicated figure with 18 elements initially, and then use their memory to draw the same figure [4]. It is frequently used to evaluate neuropsychiatric illnesses' visuo-constructional capacity and visual memory, including copying and recall tasks. Drawing the complicated figure can test a patient's functional deterioration in various cognitive domains, such as attention and concentration, fine-motor coordination, visuospatial perception, nonverbal memory, planning and organization, and spatial orientation [7]. Fig. 1 shows the completed ROCFT figure and explains how the score is given based on what is drawn by the subject.

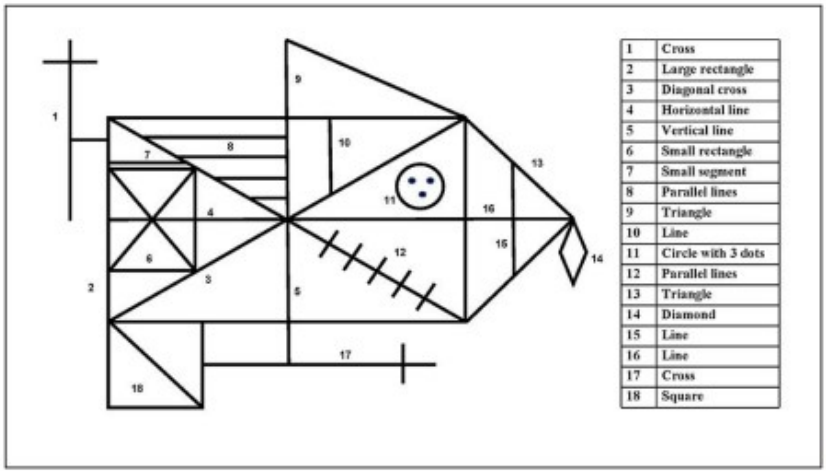

Fig. 1: Example of ROCFT scoring.

The presented study focuses on analyzing these drawings by extracting the topological features from them and further evaluating whether these topological features are suitable for the automatic scoring of these drawings.

2 Dataset

The dataset used in this project consists of real-time drawings of the Rey-Osterrieth Complex Figure done by individual subjects. As shown in the Fig. 2, the dataset images have already been pre-processed into readable grayscale images. The dataset also consists of the hand-evaluated figure scores saved in CSV format separately.

The data has been further divided into two sets, one set containing the images and scores that the subjects have copied while looking at the actual complex figure, and the other set containing the recalled images and their respective scores.

For computational efficiency, the images have been downscaled to 40 percent of their original sizes. The original width and height of an image in the dataset introduced to the model were 354*500. In total, the dataset contains 416 images where 208 are copied images and 208 are recalled images, and lastly, 208 CSV files containing the copied and recalled scores of each respective image.

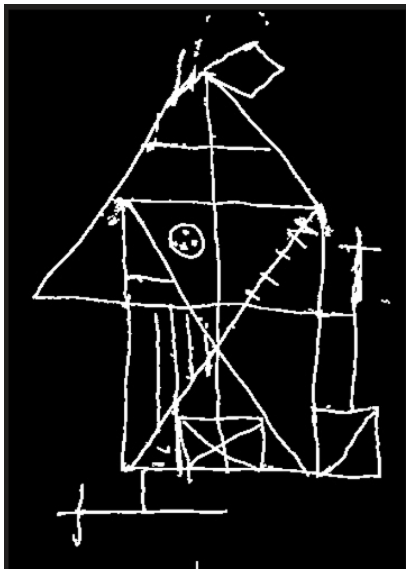

Fig. 2: Sample image from dataset.

3 Methodology

In this project, we aimed to design a data pipeline for the scoring of complex figures, using the Giotto-TDA library. The main libraries used for image processing and analysis are PIL (Python Imaging Library) and OpenCV, and various modules from gtda (Giotto-TDA) have been used for topological data analysis.

Topological data analysis (TDA) uses tools from algebraic and combinatorial topology to extract features that capture the shape of data[3]. Giotto-TDA inherits the flexibility of scikit-learn, the most popular all-purpose ML framework[5], and extends it with TDA capabilities that include a wide range of persistent homology and Mapper-type algorithms. It enables TDA to be applied to univariate and multivariate time series, images, graphs, and their higher dimensional analogs, simplicial complexes. This makes giotto-tda the most comprehensive Python library for topological machine learning and data exploration to date. Fig. 3 shows a brief comparison between Giotto-TDA and the other available similar libraries.

In TDA, persistent homology is one of the primary tools. It takes into account higher-order connectivity while extracting and summarizing multi-scale relational data in what are known as persistence diagrams, which are akin to hierarchical clustering [2]. In order to optimize its use for machine learning and data exploration tasks, giotto-tda provides scikit-learn-compatible components that allow the user to: a) convert a broad range of data input formats into for-

mats appropriate for computing persistent homology; b) compute persistence diagrams using a multitude of different algorithms; and c) extract a rich feature set from persistence diagrams. As a result, a framework for building end-to-end Pipeline objects that can produce precisely designed topological characteristics from each sample in a collection of raw input data is produced [6].

	ML framework integration	Preprocessing					Persistent homology			Diagrams features	Plotting
		Point clouds	Time series	Networks	Images	Persistent diagrams	Simplicial (undirected)	Simplicial (directed)	Cubical		
giotto-tda	✓✓	✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓	✓✓
GUDHI	✓	✓✓	✓	✗	✗	✓✓	✓✓	✗	✓✓	✓✓	✓
scikit-tda	✓	✗	✗	✗	✗	✗	✓	✗	✗	✓	✓
Dionysus 2	✗	✗	✗	✗	✗	✗	✓	✗	✗	✓	✓

Fig. 3: Giotto TDA in comparison with other libraries.

We start with data loading and preprocessing of images using the glob module. The images are then further resized to 40 percent of their original size and the image pixels are flattened as shown in Fig. 4, for better computational efficiency.

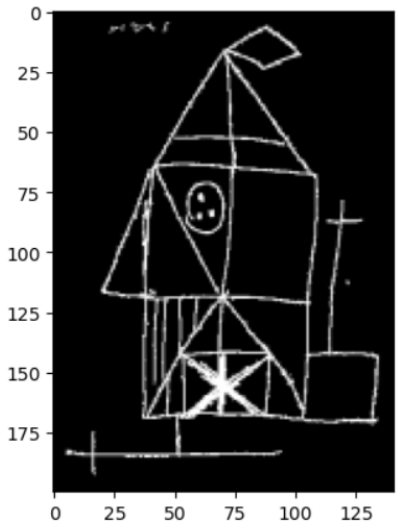

Fig. 4: Resized image.

In the next steps, persistent homology-based TDA was applied to the flattened and resized images. It uses the Binarizer class to binarize the images and then constructs persistence diagrams using CubicalPersistence. Binarization helps in simplifying the data by reducing it to binary form and enhancing the contrast.

The amplitude is calculated for each persistence diagram using different metrics, forming a feature vector for each image. For, feature selection and dimensionality reduction, the steps for persistence entropy and amplitude have been performed.

Fig. 5 shows the list of directions which has been considered for the height filtration to detect features in these directions. Similarly, Fig. 6 shows the metric list of parameters and their sub-parameters that have been fed for amplitude filtration.

```
direction_list = [[1, 0], [1, 1], [0, 1], [-1, 1], [-1, 0], [-1, -1], [0, -1], [1, -1]]
```

Fig. 5: Height Filtration Parameters.

```
metric_list = [
    {"metric": "landscape", "metric_params": {"p": 1, "n_layers": 1, "n_bins": 100}},
    {"metric": "landscape", "metric_params": {"p": 1, "n_layers": 2, "n_bins": 100}},
    {"metric": "landscape", "metric_params": {"p": 2, "n_layers": 1, "n_bins": 100}},
    {"metric": "landscape", "metric_params": {"p": 2, "n_layers": 2, "n_bins": 100}},
    {"metric": "bottleneck", "metric_params": {}},
    {"metric": "wasserstein", "metric_params": {"p": 1}},
    {"metric": "wasserstein", "metric_params": {"p": 2}},
    {"metric": "betti", "metric_params": {"p": 1, "n_bins": 100}},
    {"metric": "betti", "metric_params": {"p": 2, "n_bins": 100}},
    {"metric": "heat", "metric_params": {"p": 1, "sigma": 1.6, "n_bins": 100}},
    {"metric": "heat", "metric_params": {"p": 1, "sigma": 3.2, "n_bins": 100}},
    {"metric": "heat", "metric_params": {"p": 2, "sigma": 1.6, "n_bins": 100}},
    {"metric": "heat", "metric_params": {"p": 2, "sigma": 3.2, "n_bins": 100}},
]
```

Fig. 6: Amplitude Filtration Parameters.

Finally, Fig. 7 shows the complete pipeline with the subsequent steps that have been performed throughout for feature selection.

Furthermore, for training the model the dataset has been divided into training and testing sets, with the test set containing 20 percent of the data. The neural network model has been used for the model training. A simple feedforward neural network model is defined using TensorFlow / Keras. The defined model uses three layers with ReLU activation functions, dropout layers for regularization, and a single output layer with ReLU activation. Since ReLU doesn't

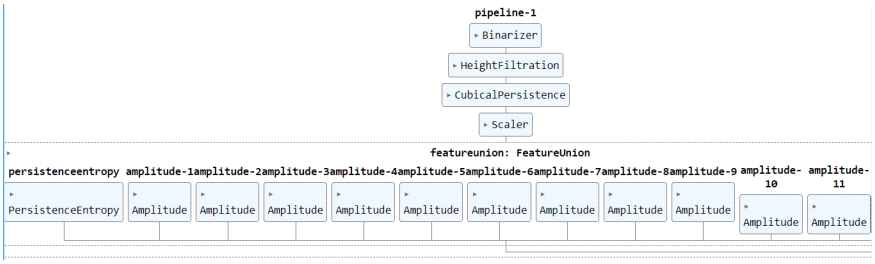

Fig. 7: Pipeline.

allow for the activation of all of the neurons at the same time, it makes our complicated network easy for computation. The model is compiled with the mean squared error loss function and mean absolute error as the metric.

4 Results

As mentioned above, the model used for the training is a regression model. The test mean absolute error recorded for the designed model is 4.9, which was trained over 20 epoch cycles.

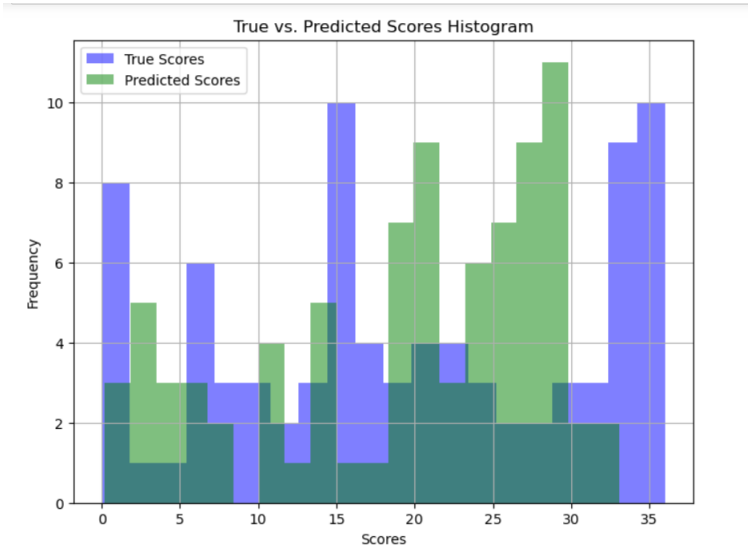

Fig. 8: Histogram of True vs Predicted scores.

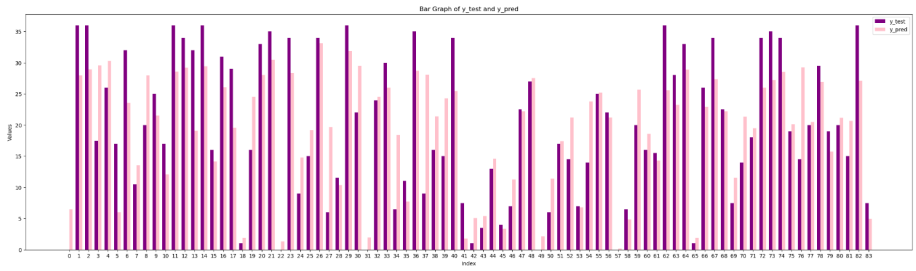

Fig. 9: Comparing True vs Predicted test scores by model.

The median absolute error recorded for the same is 4.4. The range of scores can go from the lowest 0 to the highest of 36. To understand the result better we tried to generate a few visualizations for the actual test data and what is predicted by the model itself.

So Fig.8 shows a histogram of true and predicted scores, and Fig. 9 shows a bar graph comparing true and predicted scores for each data point, to further analyze the output generated by the model for test data.

5 Conclusions

As it can be seen from the result the model behaves quite well for the middle range scores. The model behaves comparatively well in prediction for the scores from low to mid-range but not too well when it goes to higher scores above 30. The reason for this is although we have a high range of score data coming from the copying task, once the data from the recalled task is also introduced it has more scores in the lower or mid-range. Hence, we can see that most of the predicted scores are between the range of 17 to 30 as shown in Fig 8.

The trend that was observed in the data while introducing the dataset was, when first only copied data introduced to the model it behaved well for the range from mid to higher scores. We can observe from the histogram (Fig. 10) and bar graph (Fig. 11) that for the higher values the model predicts the value very near to the actual one. The reason is that since it´s copied data (where the subjects had to draw the complex figures by looking at it) the scores are mostly in the higher range. The mean absolute error observed for this was 8.4 (Fig. 12), as the dataset has fewer observations for the lower score range.

Later when the recalled images (which subjects had to draw by recalling the earlier complex figure in their memory) were introduced to the model. And as we have seen in the result the performance got much better as the recorded mean

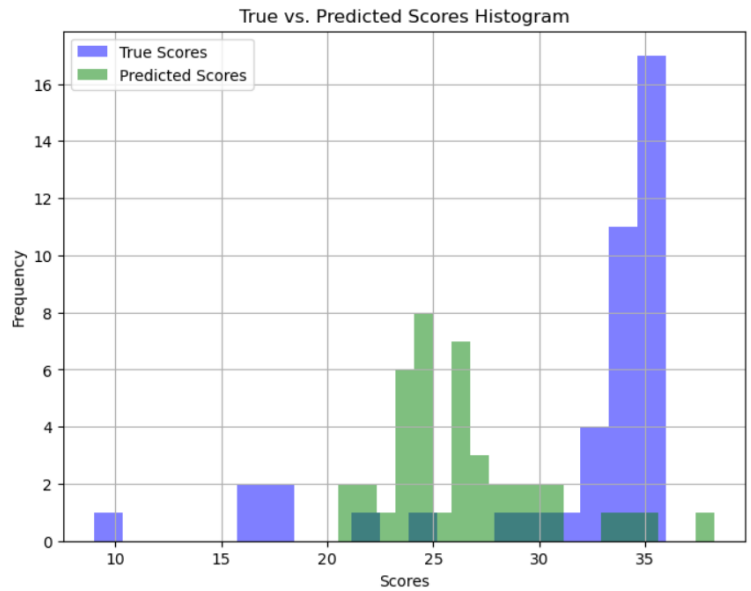

Fig. 10: Histogram of True vs Predicted scores when only copied images are considered.

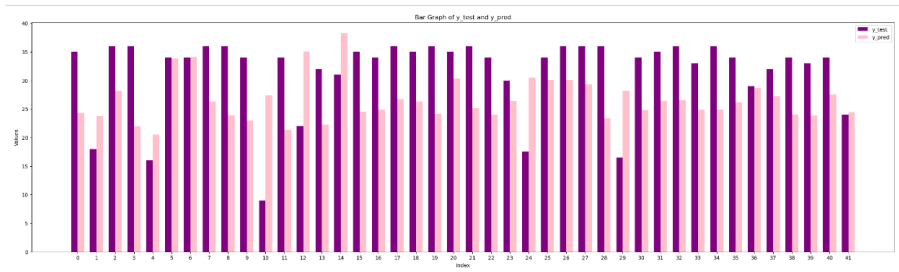

Fig. 11: Comparing True vs Predicted test scores by the model when only copied images are considered.

```
Epoch 18/20
6/6 [==============================] - 0s 4ms/step - loss: 125.3469 - mean_absolute_error: 9.1509 - val_loss: 87.4272 - val_mea
n_absolute_error: 8.3932
Epoch 19/20
6/6 [==============================] - 0s 3ms/step - loss: 145.9049 - mean_absolute_error: 9.7439 - val_loss: 90.2143 - val_mea
n_absolute_error: 8.5528
Epoch 20/20
6/6 [==============================] - 0s 3ms/step - loss: 128.5950 - mean_absolute_error: 8.9258 - val_loss: 87.9517 - val_mea
n_absolute_error: 8.4417
2/2 [==============================] - 0s 0s/step - loss: 87.9517 - mean_absolute_error: 8.4417
Test Mean Absolute Error: 8.441739082336426
```

Fig. 12: Test mean absolute error output by model when only copied images are considered.

absolute error now was 4.9. This is because now there are data for the other score range as well and the model is not just over-fitting to higher values.

So, considering all the above-mentioned observations, the regressor has the scope to perform better if there are more datasets where the score values are spread over for all value ranges instead of being just accumulated around just the lower or higher values.

Lastly, if we want to dig deeper the distribution of scores also gives an idea of what category the subjects fall into. For example, if we analyze the trend separately for the copied and the recalled one and the data is mostly fixed over a certain range of scores then we may predict that the chances are the subject group considered for the observation mostly had a specific neuropsychological dysfunction. But, that should only be taken as a possibility and a more concrete analysis should be taken out by introducing more varied ranging scores to the model.

References

1. Neuropsychological disorders. Journal of Psychological Disorders https://openaccesspub.org/psychological-disorders/neuropsychological-disorders
2. Asirimath, C., Ratnayake, J., Weeraddana, C., Rajatheva, N.: Critical points to determine persistence homology. IEEE (2020), https://ieeexplore.ieee.org/document/9048780
3. Carlsson, G.: Topology and data. American Mathematical Society **46**(2), 255–308 (2009)
4. Dimitrov, I., Kirkova, V., Kaprelyan, A., Ivanov, B., Usheva, N., Grudkova, M., Arabadzhieva, D., Deleva, N.: Application of the rey-osterrieth complex figure test for assessment of cognitive impairment in multiple sclerosis. Scripta Scientifica Medica **47**(3), 59–64 (2015)
5. Pedregosa, F., Varoquaux, G., Gramfort, A.: Machine learning in python. Journal of Machine Learning Research **12**, 2825–2830 (2011)
6. Tauzin, G., Lupo, U., Tunstall, L., P´erez, J.B., Caorsi, M., Medina-Mardones, A.M., Dassatti, A., Hess, K.: giotto-tda: A topological data analysis toolkit for machine learning and data exploration. Journal of Machine Learning Research **22**, 1–6 (2021)
7. Zhang, X., Lv, L., Min, G., Wang, Q., Zhao, Y., Li, Y.: Overview of the complex figure test and its clinical application in neuropsychiatric disorders, including copying and recall. Front Neurol (2021), https://www.frontiersin.org/journals/neurology/articles/10.3389/fneur.2021.680474/full